# TP n 2

Requêtes, dépendances fonctionnelles et normalisation

AZZOUNI Mohamed Aziz

27 mars 2026

## Ex1

**Question 1.** Afficher le nom du département qui a le budget le plus élevé.

```sql
SELECT dept_name
FROM department
WHERE budget IN (SELECT MAX(budget) FROM department);
```

**Question 2.** Afficher les salaires et les noms des enseignants qui gagnent plus que le salaire moyen.

```sql
SELECT A.name, A.salary
FROM teacher AS A
WHERE A.salary > (SELECT AVG(B.salary) FROM teacher AS B);
```

**Question 3.** Pour chaque enseignant, Afficher tous les étudiants qui ont suivi plus de deux cours dispensés par cet enseignant ainsi que le nombre total de cours, avec HAVING.

```sql
SELECT teacher.name, student.name, COUNT(*)
FROM teacher, student, takes, teaches
WHERE teacher.id = teaches.id
  AND student.id = takes.id
  AND (takes.course_id, takes.sec_id, takes.semester, takes.year)
    = (teaches.course_id, teaches.sec_id, teaches.semester, teaches.year
      )
GROUP BY teacher.name, student.name
HAVING COUNT(*) >= 2;
```

**Question 4.** Même requête sans HAVING.

```sql
SELECT T.teachername, T.studentname, T.totalcount
FROM (SELECT teacher.name AS teachername, student.name AS studentname,
              COUNT(*) AS totalcount
      FROM teacher, student, takes, teaches
       WHERE teacher.id = teaches.id
         AND student.id = takes.id
         AND (takes.course_id, takes.sec_id, takes.semester, takes.year)
           = (teaches.course_id, teaches.sec_id, teaches.semester,
              teaches.year)
       GROUP BY teacher.name, student.name) AS T
WHERE T.totalcount >= 2
ORDER BY T.teachername;
```

**Question 5.** Afficher les identifiants et les noms des étudiants qui n'ont pas suivi de cours avant 2010.

```
(SELECT student.id, student.name FROM student)
EXCEPT
(SELECT student.id, student.name
 FROM student, takes
 WHERE takes.id = student.id AND takes.year < 2010);
```

**Question 6.** Afficher tous les enseignants dont les noms commencent par E.

```
SELECT * FROM teacher WHERE name LIKE 'E%';
```

**Question 7.** Afficher les salaires et les noms des enseignants qui perçoivent le quatrième salaire le plus lev .

```
SELECT name
FROM teacher AS T1
WHERE 3 = (SELECT COUNT(DISTINCT T2.salary)
           FROM teacher AS T2
           WHERE T2.salary > T1.salary);
```

**Question 8.** Afficher les noms et les salaires des trois enseignants qui perçoivent les salaires les moins lev s, par ordre d croissant.

```
SELECT T1.name, T1.salary
FROM teacher AS T1
WHERE 2 >= (SELECT COUNT(DISTINCT T2.salary)
            FROM teacher AS T2
            WHERE T2.salary < T1.salary)
ORDER BY T1.salary ASC;
```

**Question 9.** Afficher les noms des étudiants ayant suivi un cours en automne 2009 clause IN.

```
SELECT S.name
FROM student AS S
WHERE ('Fall', 2009) IN (SELECT semester, year
                         FROM takes
                         WHERE takes.id = S.id);
```

**Question 10.** Afficher les noms des étudiants ayant suivi un cours en automne 2009 clause SOME.

```
SELECT S.name
FROM student AS S
WHERE ('Fall', 2009) = SOME (SELECT semester, year
                             FROM takes
                             WHERE takes.id = S.id);
```

**Question 11.** Afficher les noms des étudiants ayant suivi un cours en automne 2009 NATURAL INNER JOIN.

```
SELECT DISTINCT student.name
FROM takes NATURAL INNER JOIN student
WHERE takes.semester = 'Fall' AND takes.year = 2009;
```

**Question 12.** Afficher les noms Étudiants ayant suivi un cours en automne 2009 EXISTS.

```sql
SELECT name
FROM student
WHERE EXISTS (SELECT *
              FROM takes
              WHERE takes.id = student.id
                AND semester = 'Fall'
                AND year = 2009);
```

**Question 13.** Afficher toutes les paires d'étudiants qui ont suivi au moins un cours ensemble.

```sql
SELECT A.name, B.name
FROM (student NATURAL INNER JOIN takes) AS A,
     (student NATURAL INNER JOIN takes) AS B
 WHERE A.course_id = B.course_id
   AND A.sec_id = B.sec_id
   AND A.semester = B.semester
   AND A.year = B.year
   AND A.id <> B.id
   AND A.name < B.name
 GROUP BY A.id, B.id
HAVING COUNT(*) >= 1;
```

**Question 14.** Pour chaque enseignant ayant assuré un cours : nombre total d'étudiants, ordre d croissant.

```sql
SELECT teacher.name, COUNT(*)
FROM (takes INNER JOIN teaches USING (course_id, sec_id, semester, year)
    )
      INNER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(*) DESC;
```

**Question 15.** Pour chaque enseignant, même sans cours : nombre total d' étudiants.

```sql
SELECT teacher.name, COUNT(course_id)
FROM (takes INNER JOIN teaches USING (course_id, sec_id, semester, year)
    )
      RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(course_id) DESC;
```

**Question 16.** Pour chaque enseignant : nombre total de grades A attribués.

```sql
WITH mytakes(id, course_id, sec_id, semester, year, grade) AS (
     SELECT id, course_id, sec_id, semester, year, grade
     FROM takes WHERE grade = 'A')
SELECT teacher.name, COUNT(course_id)
FROM (mytakes INNER JOIN teaches USING (course_id, sec_id, semester,
    year))
      RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(course_id) DESC;
```

**Question 17.** Toutes les paires enseignant- élève et le nombre de cours suivis ensemble.

```sql
SELECT teacher.name, student.name, COUNT(*)
FROM (teacher NATURAL JOIN teaches)
```

```
      INNER JOIN (takes NATURAL JOIN student)
      USING (course_id, sec_id, semester, year)
GROUP BY teacher.name, student.name;
```

**Question 18.** Paires enseignant- élève ou l' élève a suivi au moins deux cours de l'enseignant.

```
SELECT mytable.tn, mytable.sn
FROM (SELECT teacher.name AS tn, student.name AS sn, COUNT(*) AS tc
      FROM (teacher NATURAL JOIN teaches)
            INNER JOIN (takes NATURAL JOIN student)
            USING (course_id, sec_id, semester, year)
      GROUP BY teacher.name, student.name) AS mytable
WHERE tc >= 2;
```

## Ex 2

**Question 1.** R(A, B, C) et F = {A → B; B → C}.

Cl candidate. Calculons $A^+$ : A → B donc $A^+ \supseteq \{A, B\}$, puis B → C donc $A^+ = \{A, B, C\}$. Ainsi A est l'unique cl candidate.

V ri cation BCNF :

A → B : $A^+ = \{A, B, C\}$ contient tous les attributs ⇒ A est super-cl . ✓

B → C : $B^+ = \{B, C\}$ ne contient pas A ⇒ B n'est pas super-cl . Violation BCNF.

D composition BCNF (sur la violation B → C) :

$R_1(B, C)$ avec $F_1 = \{B \to C\}$ $R_2(A, B)$ avec $F_2 = \{A \to B\}$

Les deux sous-relations sont en BCNF. La décomposition est sans perte car $R_1 \cap R_2 = \{B\}$ et B est cl de $R_1$.

**Question 2.** R(A, B, C) et F = {A → C; A → B}.

Cl candidate. $A^+ = \{A, B, C\} = R$ ⇒ A est l'unique cl candidate.

V ri cation BCNF : les deux DF ont A comme d terminant, et A est super-cl ⇒ toutes les DF respectent BCNF.

Conclusion : R est d j en BCNF (et donc en 3NF). Aucune d composition n'est nécessaire.

**Question 3.** R(A, B, C) et F = {AB → C; C → B}.

Cl s candidates.

$\{A, B\}^+ = \{A, B, C\} = R$ et aucun sous-ensemble strict n'est super-cl ⇒ {A, B} est cl candidate.

$\{A, C\}^+$ : C → B donc $\{A, C\}^+ = \{A, B, C\} = R$ ⇒ {A, C} est cl candidate.

Attributs premiers : A, B, C (tous présents dans une cl candidate).

V ri cation BCNF :

$AB \to C$ : $\{A, B\}$ est super-cl . ✓

$C \to B$ : $C^+ = \{B, C\} = R \Rightarrow C$ n'est pas super-cl . Violation BCNF.

V ri cation 3NF : B est attribut premier ($B \in \{A, B\}$) $\Rightarrow C \to B$ ne viole pas la 3NF.

Conclusion : R est en 3NF mais pas en BCNF. C'est la forme normale la plus avanc e atteignable sans perte de d pendances.

## Ex3

### 3.1 Question 1 $R(A, B, C, D, E)$, $F = \{A \to BC,\ CD \to E,\ B \to D,\ E \to A\}$

16 dépendances d duites par les axiomes d'Armstrong :

R exivit ($X \supseteq Y \Rightarrow X \to Y$) :

(a) $A \to A$

(b) $BC \to B$

(c) $BC \to C$

(d) $CD \to C$

(e) $CD \to D$

Augmentation ($X \to Y \Rightarrow XZ \to YZ$) :

(f) $AB \to BC$ (augmentation de $A \to B$ par B)

(g) $AC \to BC$ (augmentation de $A \to B$ par C)

(h) $AD \to BCD$ (augmentation de $A \to BC$ par D)

(i) $BD \to D$ (r exivit + augmentation)

(j) $ACD \to BCE$ (augmentation de $CD \to E$ avec A, combin $A \to BC$)

Transitivit ($X \to Y,\ Y \to Z \Rightarrow X \to Z$) :

(k) $A \to D$ ($A \to B$ et $B \to D$)

(l) $A \to BD$ ($A \to B$ et $B \to D$, union)

(m) $A \to BCD$ ($A \to BC$ et $A \to D$)

(n) $CD \to A$ ($CD \to E$ et $E \to A$)

(o) $CD \to BC$ ($CD \to A$ et $A \to BC$)

(p) $E \to BC$ ($E \to A$ et $A \to BC$)

| | | |
|---|---|---|
| (q) | $E \to D$ | ($E \to A$ et $A \to D$) |
| (r) | $E \to BCD$ | ($E \to BC$ et $E \to D$) |
| (s) | $B \to D$ | (donn rappel) |
| (t) | $A \to BCDE$ | ($A \to BCD$ et $CD \to E$) |

## 3.2 Question 2 $R(A, B, C, D, E, F)$, $F = \{A \to BCD,\ BC \to DE,\ B \to D,\ D \to A\}$

(a) Fermeture de $B$ et de $\{A, B\}$.

$B^+$ : On part de $\{B\}$.

$$B \to D \Rightarrow B^+ = \{B, D\}$$
$$D \to A \Rightarrow B^+ = \{A, B, D\}$$
$$A \to BCD \Rightarrow B^+ = \{A, B, C, D\}$$
$$BC \to DE \Rightarrow B^+ = \{A, B, C, D, E\}$$

$\boxed{B^+ = \{A, B, C, D, E\}}$ ($F$ absent : $B$ n'est pas super-cl de $R$.)

$\{A, B\}^+$ : On part de $\{A, B\}$.

$$A \to BCD \Rightarrow \{A, B, C, D\}$$
$$BC \to DE \Rightarrow \{A, B, C, D, E\}$$

$\boxed{\{A, B\}^+ = \{A, B, C, D, E\}}$ ($F$ absent : $\{A, B\}$ n'est pas super-cl de $R$.)

(b) $\{A, F\}$ est une super-cl .

$$A \to BCD \Rightarrow \{A, B, C, D, F\}$$
$$BC \to DE \Rightarrow \{A, B, C, D, E, F\} = R$$

$\{A, F\}^+ = R \Rightarrow \{A, F\}$ est bien une super-cl . ✓

(c) $R$ est-elle en BCNF ?

$F$ n'appara t dans aucun membre des DF, donc $F$ doit appartenir toute cl candidate.

Cl s candidates contenant $F$ :

$\{B, F\}^+ = \{A, B, C, D, E, F\} = R$ et $B^+ = R$, $F^+ = \{F\} = R \Rightarrow \{B, F\}$ est cl candidate.

$\{D, F\}^+$ : $D \to A \to BCD$, $BC \to DE \Rightarrow \{D, F\}^+ = R \Rightarrow \{D, F\}$ est cl candidate.

Violations BCNF :

$D \to A$ : $D^+ = \{A, B, C, D, E\} = R$ ($F$ manque) $\Rightarrow D$ n'est pas super-cl . Violation.

$B \to D$ : $B^+ = \{A, B, C, D, E\} = R \Rightarrow B$ n'est pas super-cl . Violation.

D composition BCNF :

1. Violation $B \to D$ : $R_1(B, D)$ (cl : $B$), $R_2(A, B, C, E, F)$.

2. Dans $R_2$, violation $A \to BC$ ($A^+|_{R_2} = \{A, B, C, E\} = R_2$ car F absent) : $R_{21}(A, B, C, E)$, $R_{22}(A, F)$.
3. $R_{21}$ : DF projet es $A \to BC$, $BC \to E$. Cl : $\{A\}$. En BCNF. ✓

   D composition nale en BCNF : $R_1(B, D)$, $R_{21}(A, B, C, E)$, $R_{22}(A, F)$.

### 3.3 Question 3 D compositions avec/sans perte

(a) $R_1(A, B, C)$ et $R_2(A, D, E)$ sans perte d'information.
$R_1 \cap R_2 = \{A\}$. D'après F, $A \to BCD$ donc $A \to \{B, C\} = R_1 \setminus \{A\}$, i.e. $R_1 \cap R_2 \to R_1$.
La condition de lossless-joinest satisfaite : la d composition est sans perte.

(b) $R_1(A, B, C)$ et $R_2(C, D, E)$ avec perte d'information.
$R_1 \cap R_2 = \{C\}$. Pour une d composition sans perte, il faudrait $C \to R_1$ ou $C \to R_2$.

$C \to \{A, B\}$ ? $C^+ = \{C\}$ (aucune DF ne part de C seul) $\Rightarrow$ Non.

$C \to \{D, E\}$ ? Idem, $C^+$ ne contient pas D ni E $\Rightarrow$ Non.

La jointure $R_1 \bowtie R_2$ produit des tuples parasites : la d composition est avec perte.

## Ex4

### Question 1. Afficher les dépendances fonctionnelles.

```
def print Dependencies (F: "list of dependencies"):
    for alpha, beta in F:
        print ("\t", alpha, " --> ", beta)
```

### Question 2. Afficher les relations.

```
def print Relations (T: "list of relations"):
    for R in T:
        print ("\t", R)
```

### Question 3. Ensemble des parties (powerset).

```
import itertools

def powerSet (inputset : "set"):
    _result = []
    for r in range (1, len(inputset) + 1):
        _result += map(set, itertools.combinations (inputset, r))
    return _result
```

### Question 4. Fermeture d'un ensemble d'attributs K.

```
def computeAttributeClosure (F: "list of dependencies", K: "set of
    attributes"):
    K_plus, size = set(K), 0
    while size != len(K_plus):
        size = len(K_plus)
        for alpha, beta in F:
            if alpha.issubset (K_plus):
                K_plus.update (beta)
```

```
    return K_plus
```

### Question 5. Fermeture de F.

```
def compute Dependencies Closure (F: "list of dependen cies "):
    R = set ()
    for alpha , beta in F:
        R. update (alpha | beta )
    F_plus = []
    for K in power Set (R):
        for beta in power Set (compute Attribute Closure (F, K)):
            F_plus . append ([ K, beta ])
    return F_plus
```

### Question 6. α d termine-t-il fonctionnellement β ?

```
def isDependency (F: "list of dependencies ",
                    alpha : "set of attributes ",
                   beta :  "set of attributes "):
    return beta . issubset (compute Attribute Closure (F, alpha ))
```

### Question 7. K est-il une super-cl ?

```
def isSuper Key (F: "list of dependencie s ",
                 R: "set defining a relation ",
                 K: " set of attributes "):
    return R. issubset ( compute Attribute Closure (F, K))
```

### Question 8. K est-il une cl candidate ?

```
def isCandidate Key (F: "list of dependencies ",
                     R: "set defining a relation ",
                     K: " set of attributes "):
    if not isSuper Key (F, R, K):
        return False
    for A in K:
        _K1 = set (K)
        _K1 . discard (A)
        if isSuper Key (F, R, _K1 ):
            return False
    return True
```

### Question 9. Toutes les cl s candidates.

```
def compute All Candidate Keys (F: "list of dependencie s ",
                                R: "set defining a relation "):
    result = []
    for K in power Set (R):
        if isCandidate Key (F, R, K):
            result . append ( K)
    return result
```

### Question 10. Toutes les super-cl s.

```
def compute All Super Keys (F: "list of dependencies ",
                            R: "set defining a relation "):
    result = []
    for K in power Set (R):
```

```
        if isSuperKey(F, R, K):
            result.append(K)
    return result
```

### Question 11. Une cl candidate.

```
def computeOneCandidateKey(F: "list of dependencies",
                           R: "set defining a relation"):
    K = set(R)
    while not isCandidateKey(F, R, K):
        for A in K:
            if isSuperKey(F, R, K.difference({A})):
                K.remove(A)
                break
    return K
```

### Question 12. R est-elle en BCNF ?

```
def isBCNFRelation(F: "list of dependencies",
                   R: "set defining a relation"):
    for K in powerSet(R):
        K_plus = computeAttributeClosure(F, K)
        Y = K_plus.difference(K)
        if not R.issubset(K_plus) and not Y.isdisjoint(R):
            return False, [K, Y & R]
    return True, [{}, {}]
```

### Question 13. Le schma T est-il en BCNF ?

```
def isBCNFRelations(F: "list of dependencies",
                    T: "list of relations"):
    for R in T:
        if isBCNFRelation(F, R) == False:
            return False, R
    return True, {}
```

### Question 14. Algorithme de d composition en BCNF.

```
def computeBCNFDecomposition(F: "list of dependencies",
                             T: "list of relations"):
    OUT, size = list(T), 0
    while size != len(OUT):
        size = len(OUT)
        for R in OUT:
            _isR_BCNF, [alpha, beta] = isBCNFRelation(F, R)
            if _isR_BCNF == False:
                if alpha | beta not in OUT:
                    OUT.append(alpha | beta)
                if R.difference(beta) not in OUT:
                    OUT.append(R.difference(beta))
                OUT.remove(R)
                break
    return OUT
```